

Καθηγητής Π. Λουρίδας

Τμήμα Διοικητικής Επιστήμης και Τεχνολογίας

Οικονομικό Πανεπιστήμιο Αθηνών

Κέρματα, McNuggets, Συναντήσεις

Το πρόβλημα των κερμάτων (coin problem), επίσης γνωστό ως το πρόβλημα των κερμάτων του Frobenius (Frobenius coin problem) ή απλούστερα το πρόβλημα του Frobenius (Frobenius problem) ρωτά ποιο είναι το μεγαλύτερο ποσό που δεν μπορούμε να πληρώσουμε χρησιμοποιώντας κέρματα με επιλεγμένες αξίες. Για παράδειγμα, αν έχουμε νομίσματα των 2 και των 5 σεντ, το μεγαλύτερο ποσό που δεν μπορούμε να πληρώσουμε είναι 3 σεντ. Αν τυχόν υπήρχαν κέρματα των 3 σεντ, το μεγαλύτερο ποσό που δεν θα μπορούσαμε να πληρώσουμε με κέρματα των 3 και των 5 σεντ είναι 7 σεντ, κ.ο.κ.

Η μαθηματική έκφραση του προβλήματος είναι: αν έχουμε θετικούς ακέραιους $\alpha_1, \alpha_2, \dots, \alpha_n$, τέτοιους ώστε ο μέγιστος κοινός διαιρέτης τους είναι η μονάδα, ποιος είναι ο μεγαλύτερος ακέραιος που δεν μπορεί να εκφραστεί ως ένας κωνικός συνδυασμός (conical combination) των αριθμών αυτών, δηλαδή ως άθροισμα $k_1\alpha_1 + k_2\alpha_2 + \dots + k_n\alpha_n$, όπου οι $k_1, k_2, \dots, k_n$ είναι μη αρνητικοί ακέραιοι. Ο μεγαλύτερος αυτός αριθμός ονομάζεται *αριθμός Frobenius* (Frobenius number) του συνόλου $\{\alpha_1, \alpha_2, \dots, \alpha_n\}$ και συνήθως συμβολίζεται με $g(\alpha_1, \alpha_2, \dots, \alpha_n)$.

Οι κωνικοί συνδυασμοί είναι πολλαπλάσια του μέγιστου κοινού διαιρέτη των αριθμών. Πράγματι, έστω ότι ο μέγιστος κοινός διαιρέτης είναι d . Τότε για κάθε α_i έχουμε $\alpha_i = d \cdot b_i$ για κάποιον ακέραιο b_i . Αυτό σημαίνει ότι:

$$s = k_1(db_1) + k_2(db_2) + \dots + k_n(db_n) = d \cdot (k_1b_1 + k_2b_2 + \dots + k_nb_n)$$

Επομένως, κάθε κωνικός συνδυασμός είναι πολλαπλάσιο του d . Τότε, αν $d \neq 1$, δεδωμένου ότι υπάρχουν άπειροι αριθμοί που δεν είναι πολλαπλάσια του d , δεν υπάρχει αριθμός Frobenius. Αν $d = 1$, τότε υπάρχει αριθμός Frobenius, και μάλιστα για $n = 2$ αποδεικνύεται ότι:

$$g(\alpha_1, \alpha_2) = \alpha_1\alpha_2 - \alpha_1 - \alpha_2$$

Αν $\alpha_1 = 2$ και $\alpha_2 = 3$, τότε $g(\alpha_1, \alpha_2) = 2 \cdot 3 - 2 - 3 = 6 - 2 - 3 = 1$. Επομένως, με κέρματα των 2 και 3 σεντ μπορούμε να αναπαραστήσουμε ως κωνικό άθροισμα κάθε θετικό ακέραιο εκτός από το 1.

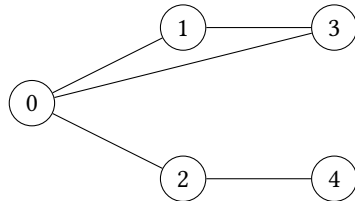
Μια ειδική περίπτωση του προβλήματος των κερμάτων είναι οι *αριθμοί McNugget* (McNugget numbers). Το πρόβλημα το εφηύρε ο Αμερικάνος μαθηματικός Henri Picciotto τη δεκαετία του 1980, όταν έτρωγε με τον γιο του σε ένα McDonald's, οπότε και το κατέγραψε σε μια χαρτοπετσέτα: ένας αριθμός McNugget είναι το σύνολο των McDonald's Chicken McNuggets που μπορούμε να έχουμε για οποιοδήποτε

αριθμό συσκευασιών. Για παράδειγμα, στο Ηνωμένο Βασίλειο, οι συσκευασίες αρχικά περιείχαν 6, 9, και 20 τεμάχια. Δεδομένου ότι ο μέγιστος κοινός διαιρέτης των 6, 9, και 20 είναι το 1, οποιοσδήποτε αρκετά μεγάλος ακέραιος μπορεί να εκφραστεί ως κωνικός συνδυασμός τους. Συγκεκριμένα, $g(6, 9, 20) = 43$.

Ας αλλάξουμε τώρα σκηνικό και ας σκεφτούμε το πρόβλημα της συνάντησης δύο ανθρώπων, της Alice και του Bob, οι οποίοι μπορούν να μετακινηθούν σε έναν χώρο που απεικονίζεται μέσω ενός γράφου. Η Alice ξεκινάει από έναν αρχικό κόμβο A και ο Bob από έναν αρχικό κόμβο B . Κάθε χρονική στιγμή και οι δυο τους μπορούν να κάνουν ένα βήμα και να μετακινηθούν σε έναν γειτονικό κόμβο. Μπορούν οι δυο τους να συναντηθούν σε έναν κόμβο στο γράφο;

Σε κάθε βήμα, οι δύο συμμετέχοντες μπορούν να μετακινηθούν σε έναν γειτονικό κόμβο από αυτόν που βρίσκονται. Μετά από k βήματα, η Alice βρίσκεται σε έναν κόμβο ο οποίος απέχει k βήματα από τον A και ο Bob βρίσκεται σε έναν κόμβο ο οποίος επίσης απέχει k βήματα από τον B . Επομένως, για να συναντηθούν, θα πρέπει να υπάρχει ένας κόμβος u τέτοιος ώστε το μονοπάτι από τον κόμβο A στον u να έχει μήκος k και το μονοπάτι από τον κόμβο B στον u να έχει μήκος k .

Πώς μπορούμε να διαπιστώσουμε αν κάτι τέτοιο είναι δυνατόν; Για να μπορεί να συμβαίνει κάτι τέτοιο, θα πρέπει να υπάρχει ένα μονοπάτι από τον κόμβο A στον κόμβο B (και το τούμπαλιν) που να έχει μήκος $2k$, δηλαδή να έχει άρτιο αριθμό βημάτων, με το σημείο συνάντησης στη μέση u , όπου και η Alice και ο Bob θα φτάσουν με τον ίδιο αριθμό βημάτων k (άρτιο ή περιττό). Εδώ τώρα πρέπει να προσέξουμε το εξής. Αν ο γράφος έχει έναν κύκλο με περιττό αριθμό βημάτων, η Alice και ο Bob μπορούν να φτάσουν σε κάθε κόμβο και με άρτιο και με περιττό αριθμό βημάτων. Δείτε για παράδειγμα τον παρακάτω γράφο:



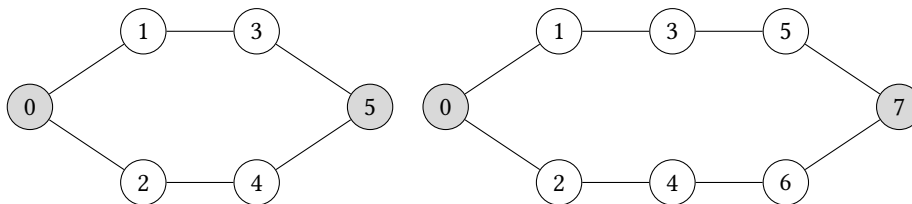
Αν ακολουθήσουμε το μονοπάτι $0 \rightarrow 1 \rightarrow 3$ και το μονοπάτι $0 \rightarrow 2 \rightarrow 4$, τότε στους κόμβους 0, 3, 4 φτάνουμε με άρτιο αριθμό βημάτων και στους κόμβους 1, 2 φτάνουμε με περιττό αριθμό βημάτων. Αλλά αν ακολουθήσουμε το μονοπάτι $0 \rightarrow 3 \rightarrow 1 \rightarrow 0 \rightarrow 2 \rightarrow 4$, φτάνουμε στους κόμβους 0, 3, 4 με περιττό αριθμό βημάτων (τη δεύτερη φορά στον 0) και στους κόμβους 1, 2 με άρτιο αριθμό βημάτων.

Αυτό μπορεί πάντα να γίνει, γιατί διαγράφοντας έναν κύκλο με περιττό μήκος αλλάζουμε την *αρτιότητα* (parity) του κάθε κόμβου που επισκεπτόμαστε. Επομένως, αν ένας γράφος έχει περιττό κύκλο, μπορούμε να πάμε από το A στο B και από το B στο A με άρτιο αριθμό βημάτων (πιθανώς χρησιμοποιώντας τον κύκλο). Και συνεπώς, σίγουρα η Alice και ο Bob μπορούν να συναντηθούν στο μέσο ενός τέτοιου μονοπατιού.

Για να μπορέσουμε να βρούμε μονοπάτια που περιλαμβάνουν έναν κύκλο, αν υπάρ-

χει, μπορούμε να χρησιμοποιήσουμε κατά πλάτος εξερεύνηση του γράφου με μία τροποποίηση. Προσθέτουμε σε κάθε κόμβο που επισκεπτόμαστε την αρτιότητα p με την οποία τον επισκεπτόμαστε, όπου $p = 0$ για άρτιο αριθμό βημάτων και $p = 1$ για περιττό αριθμό βημάτων. Έτσι, αν ξεκινάμε από τον A , έχουμε τον κόμβο $(A, 0)$. Επισκεπτόμαστε κάθε γείτονα u του A με αρτιότητα $(u, 1)$, κάθε γείτονα v του u με αρτιότητα $(v, 0)$, κ.ο.κ. Βεβαίως αν υπάρχει περιττός κύκλος θα έχουμε για κάθε κόμβο u του αρχικού γράφου και τον $(u, 0)$ και τον $(u, 1)$. Η προσθήκη της αρτιότητας ακριβώς μας επιτρέπει να επισκεφτούμε κάθε κόμβο του γράφου και μέσω κύκλου. Επομένως κάνουμε κατά πλάτος αναζήτηση όπως περιγράψαμε από τον κόμβο A , κάνουμε μια άλλη κατά πλάτος αναζήτηση από τον κόμβο B , και ελέγχουμε αν υπάρχουν κόμβοι που επισκεφτήκαμε και έχουν το ίδιο μήκος μονοπατιού από τον A και B . Αν υπάρχουν, επιστρέφουμε έναν με το μικρότερο μονοπάτι (αυτόν που βρήκαμε πρώτον).

Τα πράγματα αλλάζουν αν ο γράφος δεν έχει περιττό κύκλο, οπότε οι κατά πλάτος αναζητήσεις που περιγράψαμε δεν θα βρουν πάντα σημείο συνάντησης. Για παράδειγμα, στον γράφο κάτω αριστερά, μπορείτε να διαπιστώσετε ότι η Alice και ο Bob δεν μπορούν να συναντηθούν πουθενά, αν η Alice ξεκινάει από τον άκρο αριστερά κόμβο 0, και ο Bob από τον άκρο δεξιά κόμβο 5. Στο γράφο δεξιά όμως, αν ξεκινήσουν από τον 0 και τον 7 μπορούν να συναντηθούν στον 3 ή στον 4.

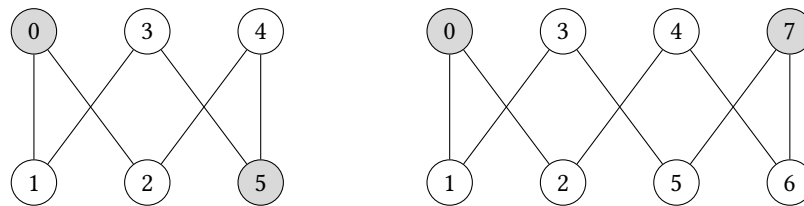


Ας θυμηθούμε εδώ τον ορισμό του *διμερή γράφου* (bipartite graph). Ένας γράφος είναι διμερής αν οι κόμβοι του μπορούν να χωριστούν σε δύο σύνολα ώστε όλοι οι σύνδεσμοι είναι μεταξύ των συνόλων. Αυτό σημαίνει ότι ένας γράφος είναι διμερής αν και μόνο αν δεν έχει περιττούς κύκλους. Πράγματι, έστω ότι ένας γράφος είναι διμερής. Τότε αν έχει κύκλους, κάθε κύκλος θα πρέπει να αποτελείται από συνδέσμους που μας μεταφέρουν από το ένα σύνολο στο άλλο. Αφού θα πρέπει να επιστρέψουμε στον αρχικό κόμβο, ο κύκλος αναγκαστικά θα αποτελείται από μία σειρά «πήγαινε-έλα» μεταξύ των δύο συνόλων κόμβων, άρα θα έχει άρτιο μήκος. Αντιστρόφως, έστω ότι ένας γράφος δεν έχει περιττούς κύκλους. Τότε μπορούμε να ξεκινήσουμε από έναν κόμβο και να διασχίσουμε το γράφο εντάσσοντας εναλλάξ τους κόμβους κατά τη διάσχιση σε δύο διαφορετικά σύνολα. Αν τυχόν συναντήσουμε έναν κόμβο τον οποίο έχουμε ήδη επισκεφτεί, αφού ο κύκλος θα είναι άρτιος, το σύνολο στο οποίο θα τον βάλουμε τη δεύτερη φορά που τον επισκεπτόμαστε θα είναι το ίδιο με αυτό στο οποίο τον βάλουμε την πρώτη. Επομένως οι κόμβοι μπορούν να ενταχθούν σε δύο διακριτά σύνολα, ώστε όλοι οι σύνδεσμοι του γράφου είναι μεταξύ των συνόλων.

Έχοντας αυτό υπόψη, συμπεραίνουμε ότι αν ο γράφος δεν είναι διμερής, τότε η Alice και ο Bob μπορούν σίγουρα να συναντηθούν, αφού ο γράφος θα έχει έναν περιττό κύκλο, οπότε εμπίπτει στην περίπτωση που περιγράψαμε. Αν λοιπόν ο γράφος δεν

έχει περιττό κύκλο, οπότε είναι διμερής, τότε διακρίνουμε δύο περιπτώσεις. Στην πρώτη περίπτωση, η Alice και ο Bob ξεκινούν και οι δύο από κόμβους στο ίδιο σύνολο. Τότε σε κάθε βήμα που προχωρούν αλλάζουν και οι δύο ταυτόχρονα σύνολο, και άρα μπορούν να συναντηθούν σε έναν κόμβο. Στη δεύτερη περίπτωση, η Alice και ο Bob ξεκινούν από κόμβους που ανήκουν σε διαφορετικά σύνολα. Τότε σε κάθε βήμα που προχωρούν αλλάζουν και οι δύο ταυτόχρονα σύνολο, και θα εξακολουθούν να βρίσκονται σε διαφορετικά, επομένως δεν θα μπορούν να συναντηθούν ποτέ.

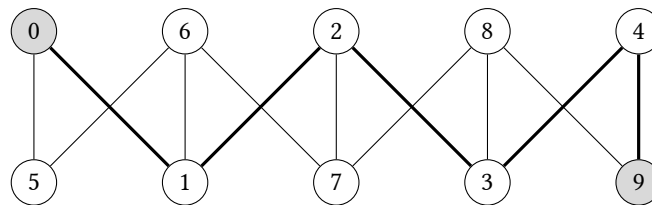
Οι δύο παραπάνω γράφοι είναι διμερείς. Ο καταμερισμός των κόμβων σε δύο σύνολα δεν φαίνεται όπως τους είχαμε αποτυπώσει, αλλά φαίνεται αν τους αποτυπώσουμε όπως παρακάτω:



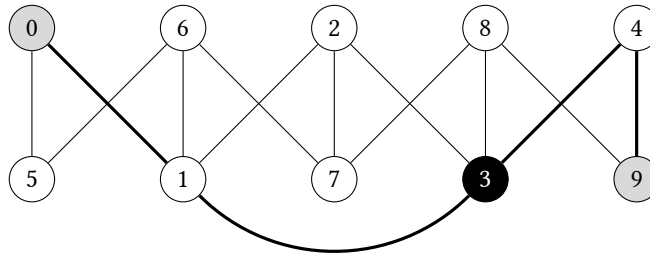
Μπορούμε να δούμε ότι στον αριστερό γράφο, με κόμβους εκκίνησης 0 και 5 δεν υπάρχει κοινός τόπος συνάντησης, αφού οι εκκινήσεις ανήκουν σε διαφορετικά σύνολα. Αντίθετα, στον δεξιό γράφο, με κόμβους εκκίνησης 0 και 7, υπάρχουν οι κόμβοι συνάντησης 3 και 4, αφού οι εκκινήσεις ανήκουν στο ίδιο σύνολο.

Το πρόβλημα λοιπόν έγκειται στις περιπτώσεις που έχουμε διμερή γράφο και οι κόμβοι εκκίνησης δεν ανήκουν στο ίδιο σύνολο. Τότε, προκειμένου να μπορούν να συναντηθούν η Alice και ο Bob, θα πρέπει να αλλάξουμε το γράφο. Θέλουμε όμως να τον αλλάξουμε με τον ελάχιστο δυνατόν τρόπο.

Στον παρακάτω γράφο, η Alice και ο Bob δεν μπορούν να συναντηθούν ως έχει. Μέσω κατά πλάτος αναζήτησης το μονοπάτι που τους συνδέει είναι το $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 9$. Μπορείτε να διαπιστώσετε ότι η Alice και ο Bob θα προσπεράσουν ο ένας τον άλλο και δεν θα συμπέσουν σε κανέναν κόμβο του μονοπατιού αυτού.

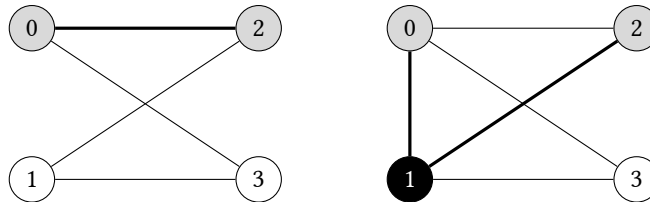


Η λύση είναι να προστεθεί ο σύνδεσμος $(1 - 3)$, οπότε η Alice και ο Bob μπορούν να συναντηθούν στον κόμβο 3 ακολουθώντας τα μονοπάτια $0 \rightarrow 1 \rightarrow 3$ και $9 \rightarrow 4 \rightarrow 3$ αντίστοιχα.



Για να βρούμε το σύνδεσμο $(1 - 3)$ που πρέπει να προσθέσουμε, εργαστήκαμε ως εξής. Βρήκαμε μέσω κατά πλάτος αναζήτησης το συντομότερο μονοπάτι από τον 0 στον 9: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 9$. Το μονοπάτι αυτό έχει έξι κόμβους, και ο κόμβος στο μέσο του είναι ο κόμβος στη θέση $\lfloor 6/2 \rfloor = 3$, δηλαδή ο κόμβος 3. Βρίσκουμε τότε τον γείτονα του 3 που προηγείται κατά δύο θέσεις, δηλαδή τον κόμβο 1, και τον ενώνουμε με το 3. Το σημείο συνάντησης είναι το μέσο του νέου μονοπατιού $0 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 9$.

Υπάρχει η (ακραία) περίπτωση οι δύο κόμβοι εκκίνησης να συνδέονται απευθείας, οπότε η παραπάνω λογική δεν εφαρμόζεται, όπως στον παρακάτω γράφο αριστερά. Μπορούμε όμως απλά να συνδέσουμε τότε τον ένα κόμβο εκκίνησης με τον μικρότερο άλλο γείτονα του άλλου κόμβου εκκίνησης, όπως φαίνεται δεξιά.



Με όσα περιγράψαμε μπορούμε πάντα να βρούμε ένα σημείο συνάντησης σε έναν μη κατευθυνόμενο γράφο. Αλλά τι γίνεται αν ο γράφος είναι κατευθυνόμενος;

Κατ' αρχήν μπορούμε να εργαστούμε όπως και στον μη κατευθυνόμενο γράφο, κάνοντας αναζήτηση στο γράφο, χρησιμοποιώντας την αρτιότητα, και από τα δύο σημεία εκκίνησης και ελέγχοντας αν υπάρχει σημείο συνάντησης. Αν δεν υπάρχει, τότε θα πρέπει να προσπαθήσουμε περισσότερο.

Χρησιμοποιώντας κατά πλάτος αναζήτηση στον αρχικό γράφο, με αφετηρία τον κόμβο A μπορούμε να βρούμε τους κόμβους που μπορεί να επισκεφτεί η Alice· αντίστοιχα, με κατά πλάτος αναζήτηση στον αρχικό γράφο με αφετηρία τον κόμβο B μπορούμε να βρούμε τους κόμβους που μπορεί να επισκεφτεί ο Bob. Εντοπίζουμε τότε τον κόμβο u με το μικρότερο άθροισμα μονοπατιών από την Alice και από τον Bob. Αν υπάρχουν περισσότεροι του ενός, επιλέγουμε τον μικρότερο.

Στη συνέχεια, εξετάζουμε αν η Alice και ο Bob μπορούν να συναντηθούν προσθέτοντας συνδέσμους που δημιουργούν κύκλους στο γράφο. Ξεκινάμε δημιουργώντας κύκλους μήκους δύο, δηλαδή δοκιμάζουμε προσθέτοντας έναν σύνδεσμο από τον u σε κάθε γείτονά του w που δείχνει στον u . Θα ονομάζουμε τους κύκλους μήκους δύο

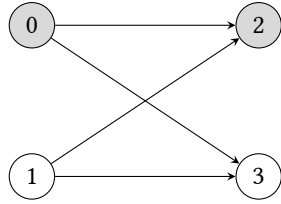
2-κύκλους. Για κάθε έναν 2-κύκλο που φτιάχνουμε, $u \rightarrow w \rightarrow u$, ελέγχουμε αν τώρα υπάρχει σημείο συνάντησης. Αν τυχόν βρούμε σημείο συνάντησης για παραπάνω από έναν κύκλο, επιλέγουμε αυτό που οι Alice και ο Bob συναντιούνται με τα λιγότερα συνολικά βήματα.

Ο εντοπισμός του σημείου συνάντησης τώρα γίνεται με μία άλλη παραλλαγή της κατά πλάτος αναζήτησης, όπου χρησιμοποιούμε έναν γράφο με κόμβους (a, b) , όπου a είναι ο κόμβος που επισκέπτεται η Alice και b είναι ο κόμβος που επισκέπτεται ο Bob. Ξεκινάμε λοιπόν από τον κόμβο (A, B) . Οι γείτονες αυτού του κόμβου είναι οι κόμβοι $(a_{1,i}, b_{1,j})$ όπου $a_{1,i}$ είναι γείτονας του A και $b_{1,j}$ είναι γείτονας του B . Οι γείτονες του κάθε $(a_{1,i}, b_{1,j})$ είναι οι κόμβοι $(a_{2,i}, b_{2,j})$ όπου ο κάθε $a_{2,i}$ είναι γείτονας ενός από τους $a_{1,i}$ και ο κάθε $b_{2,j}$ είναι γείτονας ενός από τους $b_{1,j}$. Στην ουσία οι κόμβοι είναι το καρτεσιανό γινόμενο των δυνατών καταστάσεων, δηλαδή θέσεων, στις οποίες μπορούν να βρίσκονται ταυτόχρονα η Alice και ο Bob.

Σε περίπτωση που δεν βρούμε σημείο συνάντησης με 2-κύκλους, κάνουμε την ίδια διαδικασία για κύκλους μήκους τρία, που θα τους ονομάζουμε 3-κύκλους. Βρίσκουμε όλους τους κόμβους που βρίσκονται σε απόσταση δύο συνδέσμων από τον u . Δοκιμάζουμε για κάθε έναν τέτοιο κόμβο w να προσθέσουμε ένα σύνδεσμο $u \rightarrow w$, οπότε θα προκύψει 3-κύκλος, $u \rightarrow w \rightarrow v \rightarrow u$. Για κάθε έναν τέτοιο κύκλο, ελέγχουμε αν υπάρχει σημείο συνάντησης. Αν τυχόν πάλι βρούμε σημείο συνάντησης για παραπάνω από έναν κύκλο, επιλέγουμε αυτό που οι Alice και ο Bob συναντιούνται με τα λιγότερα συνολικά βήματα.

Σε περίπτωση που δεν βρούμε σημείο συνάντησης ούτε με 2-κύκλους ούτε με 3-κύκλους, κάνουμε τη διαδικασία δοκιμάζοντας κάθε συνδυασμό 2-κύκλων και 3-κύκλων.

Υπάρχει η ακραία περίπτωση να μην μπορούμε να φτιάξουμε 3-κύκλους, όπως στον παρακάτω γράφο με κόμβους εκκίνησης τον 0 και τον 2:



Αυτό μπορούμε να το δεχτούμε ως ακραία περίπτωση· αν θέλαμε, θα μπορούσαμε απλώς να προσθέσουμε ένα σύνδεσμο $2 \rightarrow 3$ ώστε η Alice και ο Bob να συναντηθούν στον κόμβο 3 σε ένα βήμα, αλλά αν δεχτούμε κάτι τέτοιο μπορούμε πάντα να προσθέτουμε παρακάμψεις σε έναν γράφο και να συναντιούνται σε ένα βήμα.

Τι σχέση έχουν αυτά με τα κέρματα και τα McNuggets; Στην περίπτωση του κατευθυνόμενου γράφου, η Alice και ο Bob θέλουν να συναντηθούν σε έναν κόμβο u . Έστω ότι η Alice φτάνει πριν από τον Bob. Τότε αν η Alice φτάσει σε $d(A, u)$ βήματα και ο Bob φτάσει σε $d(B, u)$ βήματα, η Alice θα πρέπει με κάποιον τρόπο να περιμένει $|d(A, u) - d(B, u)|$ βήματα. Αυτό μπορεί να το κάνει κάνοντας κύκλους γύρω από τον u . Αν έχουμε κύκλους μήκους δύο και τρία τότε, όπως είπαμε παραπάνω, μπορεί

διαβαίνοντάς τους να περιμένει ακριβώς οποιονδήποτε αριθμό βημάτων μεγαλύτερο της μονάδας. Αν $|d(A, u) - d(B, u)| = 1$, τότε μπορούμε να φτιάξουμε έναν κύκλο μήκους τρία από τον u στο μονοπάτι του πιο αργού από τους δύο, έστω του Bob, οπότε ο Bob μπορεί να κάνει την παράκαμψη μέσω του συνδέσμου που προσθέσαμε για να φτιάξουμε στον κύκλο και να φτάσει στον κόμβο u σε ένα βήμα λιγότερο.

Το αντικείμενο της εργασίας είναι η υλοποίηση προγράμματος στη γλώσσα Python το οποίο θα παίρνει ως είσοδο έναν γράφο και δύο σημεία εκκίνησης και θα βρίσκει το σημείο συνάντησης της Alice και του Bob, κάνοντας τις αλλαγές που τυχόν χρειάζεται στον γράφο.

Λεπτομέρειες Υλοποίησης

Θα χρησιμοποιήσετε λίστες γειτνίασης για την αναπαράσταση του γράφου στο πρόγραμμα. Η κάθε λίστα θα περιέχει τους γείτονες ενός κόμβου ταξινομημένους σε αύξουσα σειρά.

Απαιτήσεις Προγράμματος

Κάθε φοιτητής θα εργαστεί σε αποθετήριο στο GitHub. Για να αξιολογηθεί μια εργασία θα πρέπει να πληροί τις παρακάτω προϋποθέσεις:

- Για την υποβολή της εργασίας θα χρησιμοποιηθεί το ιδιωτικό αποθετήριο του φοιτητή που δημιουργήθηκε για τις ανάγκες του μαθήματος και του έχει αποδοθεί. Το αποθετήριο αυτό έχει όνομα του τύπου `username-algo-assignments`, όπου `username` είναι το όνομα του φοιτητή στο GitHub. Για παράδειγμα, το σχετικό αποθετήριο του διδάσκοντα θα ονομαζόταν `louridas-algo-assignments` και θα ήταν προσβάσιμο στο <https://github.com/dmst-algorithms-course/louridas-algo-assignments>. Τυχόν άλλα αποθετήρια απλώς θα αγνοθούν.
- Μέσα στο αποθετήριο αυτό θα πρέπει να δημιουργηθεί ένας κατάλογος `assignment-2026-2`.
- Μέσα στον παραπάνω κατάλογο το πρόγραμμα θα πρέπει να αποθηκευτεί με το όνομα `rendezvous.py`.
- Δεν επιτρέπεται η χρήση έτοιμων βιβλιοθηκών γράφων ή τυχόν έτοιμων υλοποιήσεων των αλγορίθμων, ή τμημάτων αυτών, εκτός αν αναφέρεται ρητά ότι επιτρέπεται.
- Επιτρέπεται η χρήση δομών δεδομένων της Python όπως στοίβες, λεξικά, σύνολα, κ.λπ.
- Επιτρέπεται η χρήση των παρακάτω βιβλιοθηκών ή τμημάτων τους όπως ορίζεται:
 - `sys`
 - `deque`

– `bisect`

- Το πρόγραμμα θα πρέπει να είναι γραμμένο σε Python 3.
- Η εργασία είναι αποκλειστικά ατομική. Δεν επιτρέπεται συνεργασία μεταξύ φοιτητών στην εκπόνησή της, με ποινή το μηδενισμό. Επιπλέον η εργασία δεν μπορεί να είναι αποτέλεσμα συστημάτων Τεχνητής Νοημοσύνης. Ειδικότερα όσον αφορά το τελευταίο σημείο προσέξτε ότι τα συστήματα αυτά χωλαίνουν στην αλγοριθμική σχέση, άρα τυχόν προτάσεις που κάνουν σε σχετικά θέματα μπορεί να είναι λανθασμένες. Επιπλέον, αν θέλετε να χρησιμοποιήσετε τέτοια συστήματα, τότε αν και κάποιος άλλος το κάνει αυτό, μπορεί οι προτάσεις που θα λάβετε να είναι παρόμοιες, οπότε οι εργασίες θα παρουσιάσουν ομοιότητες και άρα θα μηδενιστούν.
- Η έξοδος του προγράμματος θα πρέπει να περιλαμβάνει μόνο ό,τι φαίνεται στα παραδείγματα που παρατίθενται.

Η φλυαρία δεν επιβραβεύεται.

Χρήση του GitHub

Όπως αναφέρθηκε το πρόγραμμά σας για να αξιολογηθεί θα πρέπει να αποθηκευθεί στο GitHub. Επιπλέον, θα πρέπει το GitHub να χρησιμοποιηθεί καθόλη τη διάρκεια της ανάπτυξης του. Θα εργαστείτε στο αποθετήριο που σας έχει αποδοθεί και:

- δεν θα κάνετε fork κάποιο άλλο αποθετήριο (όπως της εκφώνησης)
- θα εργαστείτε στο main branch
- δεν θα κάνετε pull και merge requests

Προσέξτε ότι *σίγουρα δεν θα ανεβάσετε στο GitHub απλώς την τελική λύση της εργασίας*. Στο GitHub θα πρέπει να φαίνεται το ιστορικό της συγγραφής του προγράμματος. Αυτό συνάδει και με τη φιλοσοφία του εργαλείου: λέμε “commit early, commit often”. Εργαζόμαστε σε ένα πρόγραμμα και κάθε μέρα, ή όποια στιγμή έχουμε κάνει κάποιο σημαντικό βήμα, αποθηκεύουμε την αλλαγή στο GitHub. Αυτό έχει σειρά ευεργετικών αποτελεσμάτων:

- Έχουμε πάντα ένα αξιόπιστο εφεδρικό μέσο στο οποίο μπορούμε να ανατρέξουμε αν κάτι πάει στραβά στον υπολογιστή μας (μας γλιτώνει από πανικούς του τύπου: ένα βράδυ πριν από την παράδοση ο υπολογιστής μας ή ο δίσκος του πνέει τα λούσθια, και εμείς τι θα υποβάλουμε στον Λουρίδα;).
- Καθώς έχουμε πλήρες ιστορικό των σημαντικών αλλαγών και εκδόσεων του προγράμματός μας, μπορούμε να επιστρέψουμε σε μία προηγούμενη αν συνειδητοποιήσουμε κάποια στιγμή ότι πήραμε λάθος δρόμο (μας γλιτώνει από πανικούς του τύπου: μα το πρωί δούλευε σωστά, τι στο καλό έκανα και τώρα δεν παίζει τίποτε;).

- Καθώς φαίνεται η πρόοδος μας στο GitHub, επιβεβαιώνουμε στον διδάσκοντα ότι πραγματικά έχουμε εργαστεί μεθοδικά και δεν μας δώθηκε η λύση στο πιάτο (μας γλιτώνει από πανικούς του τύπου: καλά, πώς το έλυσες το πρόβλημα αυτό με τη μία, μόνος σου;).
- Αν δουλεύετε σε μία ομάδα που βεβαίως δεν είναι καθόλου η περίπτωση μας εδώ, μπορούν όλοι να εργάζονται στα ίδια αρχεία στο GitHub εξασφαλίζοντας ότι ο ένας δεν γράφει πάνω στις αλλαγές του άλλου. Παρά το ότι η εργασία αυτή είναι ατομική, καλό είναι να αποκτάτε τριβή με το εργαλείο git και την υπηρεσία GitHub μιας και χρησιμοποιούνται ευρέως και όχι μόνο στη συγγραφή κώδικα.

Άρα επενδύστε λίγο χρόνο στην εκμάθηση των git / GitHub, των οποίων ο σωστός τρόπος χρήσης είναι μέσω γραμμής εντολών (command line), ή ειδικών προγραμμάτων (clients) ή μέσω ενοποίησης στο περιβάλλον ανάπτυξης (editor, IDE).

Όσον αφορά την αξιολόγηση της εργασίας, θα ληφθεί υπόψη ακριβώς η ιστορικότητα που θα έχει καταχωρηθεί στο GitHub. Δεν θα αξιολογηθούν εργασίες οι οποίες στο GitHub εμφανίζουν μόνο ένα commit, ή ελάχιστα commit. Η ιστορικότητα της εργασίας, όπως αποτυπώνεται στο GitHub, θα πρέπει να αντανακλά την πρόοδο στην εκπόνησή της από την αρχή της υλοποίησης μέχρι το τέλος της προθεσμίας.

Εργασίες που υποβάλλονται ως επιφοίτηση δεν θα βαθμολογούνται.

Χρήση Προγράμματος

Το πρόγραμμα θα καλείται ως εξής (όπου python η κατάλληλη εντολή στο εκάστοτε σύστημα):

```
python rendezvous.py [-d] <graph_file>
```

Το πρόγραμμα παίρνει ως υποχρεωτική παράμετρο το όνομα του αρχείου, graph_file, το οποίο περιγράφει τον γράφο. Ο γράφος περιγράφεται σε ένα αρχείο με γραμμές με δύο αριθμούς x και y.

x y

Η πρώτη γραμμή του αρχείου περιέχει τον αριθμό των κόμβων και τον αριθμό των ακμών του γράφου. Η τελευταία γραμμή του αρχείου περιέχει τους δύο κόμβους εκκίνησης, της Alice και του Bob. Οι ενδιάμεσες γραμμές περιέχουν τους συνδέσμους του γράφου.

Αν δοθεί η παράμετρος -d ο γράφος είναι κατευθυνόμενος· διαφορετικά, δεν είναι.

Παραδείγματα

Παράδειγμα 1

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py graph_1.txt
```

το πρόγραμμά σας θα διαβάσει το αρχείο [graph_1.txt](#) και θα εμφανίσει στην έξοδο:

```
0: Alice at 0, Bob at 4
1: Alice at 1, Bob at 3
2: Alice at 2, Bob at 2
Meeting at node 2 at time step 2.
```

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py -d graph_1.txt
```

το πρόγραμμα θα εμφανίσει στην έξοδο:

```
No meeting is possible.
Adding 1 edge.
Adding 4 3.
0: Alice at 0, Bob at 4
1: Alice at 1, Bob at 3
2: Alice at 2, Bob at 4
3: Alice at 3, Bob at 3
Meeting at node 3 at time step 3.
```

Παράδειγμα 2

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py graph_2.txt
```

το πρόγραμμά σας θα διαβάσει το αρχείο [graph_2.txt](#) και θα εμφανίσει στην έξοδο:

```
0: Alice at 0, Bob at 2
1: Alice at 3, Bob at 3
Meeting at node 3 at time step 1.
```

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py -d graph_2.txt
```

το πρόγραμμα θα εμφανίσει στην έξοδο:

```
No meeting is possible.
Adding 1 edge.
Adding 0 3.
0: Alice at 0, Bob at 2
1: Alice at 3, Bob at 3
Meeting at node 3 at time step 1.
```

Παράδειγμα 3

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py graph_3.txt
```

το πρόγραμμά σας θα διαβάσει το αρχείο [graph_3.txt](#) και θα εμφανίσει στην έξοδο:

```
No meeting is possible.  
Adding 1 edge.  
Adding 0 1.  
0: Alice at 0, Bob at 2  
1: Alice at 1, Bob at 1  
Meeting at node 1 at time step 1.
```

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py -d graph_3.txt
```

το πρόγραμμα θα εμφανίσει στην έξοδο:

```
No meeting is possible.  
Could not establish a rendezvous by adding edges.
```

Παράδειγμα 4

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py graph_4.txt
```

το πρόγραμμά σας θα διαβάσει το αρχείο [graph_4.txt](#) και θα εμφανίσει στην έξοδο:

```
0: Alice at 0, Bob at 1  
1: Alice at 2, Bob at 2  
Meeting at node 2 at time step 1.
```

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py -d graph_4.txt
```

το πρόγραμμα θα εμφανίσει στην έξοδο:

```
No meeting is possible.  
Adding 1 edge.  
Adding 1 0.  
0: Alice at 0, Bob at 1  
1: Alice at 1, Bob at 2  
2: Alice at 0, Bob at 0  
Meeting at node 0 at time step 2.
```

Παράδειγμα 5

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py graph_5.txt
```

το πρόγραμμά σας θα διαβάσει το αρχείο [graph_5.txt](#) και θα εμφανίσει στην έξοδο:

```
No meeting is possible.  
Adding 1 edge.  
Adding 1 3.  
0: Alice at 0, Bob at 9  
1: Alice at 1, Bob at 4  
2: Alice at 3, Bob at 3  
Meeting at node 3 at time step 2.
```

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py -d graph_5.txt
```

το πρόγραμμα θα εμφανίσει στην έξοδο:

```
No meeting is possible.  
Adding 2 edges.  
Adding 9 4.  
Adding 9 3.  
0: Alice at 0, Bob at 9  
1: Alice at 1, Bob at 4  
2: Alice at 2, Bob at 9  
3: Alice at 3, Bob at 3  
Meeting at node 3 at time step 3.
```

Παράδειγμα 6

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py graph_6.txt
```

το πρόγραμμά σας θα διαβάσει το αρχείο [graph_6.txt](#) και θα εμφανίσει στην έξοδο:

```
0: Alice at 0, Bob at 1  
1: Alice at 2, Bob at 2  
Meeting at node 2 at time step 1.
```

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py -d graph_6.txt
```

το πρόγραμμα θα εμφανίσει στην έξοδο:

```
0: Alice at 0, Bob at 1  
1: Alice at 2, Bob at 2  
Meeting at node 2 at time step 1.
```

Παράδειγμα 7

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py graph_7.txt
```

το πρόγραμμά σας θα διαβάσει το αρχείο [graph_7.txt](#) και θα εμφανίσει στην έξοδο:

```
No meeting is possible.  
Could not establish a rendezvous by adding edges.
```

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py -d graph_7.txt
```

το πρόγραμμα θα εμφανίσει στην έξοδο:

```
No meeting is possible.  
Could not establish a rendezvous by adding edges.
```

Παράδειγμα 8

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py graph_8.txt
```

το πρόγραμμά σας θα διαβάσει το αρχείο [graph_8.txt](#) και θα εμφανίσει στην έξοδο:

```
No meeting is possible.  
Adding 1 edge.  
Adding 4 3.  
0: Alice at 0, Bob at 4  
1: Alice at 3, Bob at 3  
Meeting at node 3 at time step 1.
```

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py -d graph_8.txt
```

το πρόγραμμα θα εμφανίσει στην έξοδο:

```
No meeting is possible.  
Adding 2 edges.  
Adding 4 0.  
Adding 4 3.  
0: Alice at 0, Bob at 4  
1: Alice at 4, Bob at 3  
2: Alice at 0, Bob at 0  
Meeting at node 0 at time step 2.
```

Παράδειγμα 9

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py graph_9.txt
```

το πρόγραμμά σας θα διαβάσει το αρχείο [graph_9.txt](#) και θα εμφανίσει στην έξοδο:

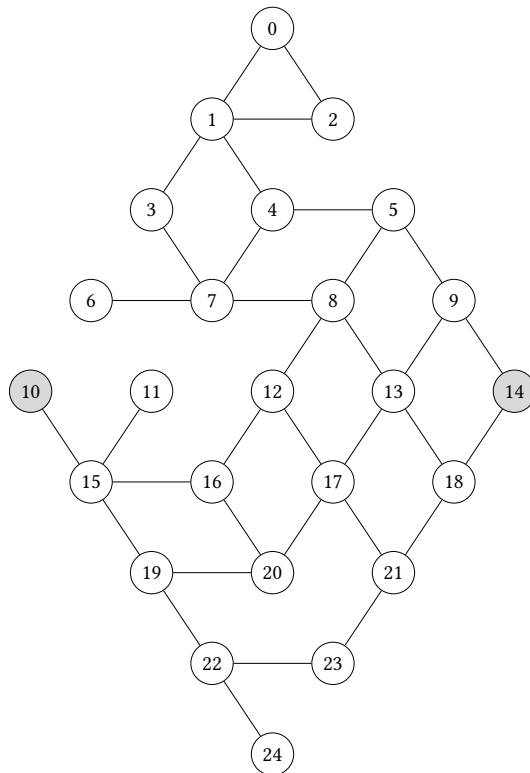
```
No meeting is possible.  
Adding 1 edge.  
Adding 0 2.  
0: Alice at 0, Bob at 1  
1: Alice at 2, Bob at 2  
Meeting at node 2 at time step 1.  
  
Αν ο χρήστης του προγράμματος δώσει:  
python rendezvous.py -d graph_9.txt  
το πρόγραμμα θα εμφανίσει στην έξοδο:  
  
No meeting is possible.  
Adding 2 edges.  
Adding 1 0.  
Adding 1 5.  
0: Alice at 0, Bob at 1  
1: Alice at 1, Bob at 5  
2: Alice at 0, Bob at 0  
Meeting at node 0 at time step 2.
```

Παράδειγμα 10

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py graph_10.txt
```

το πρόγραμμά σας θα διαβάσει το αρχείο [graph_10.txt](#), που αντιστοιχεί στον γράφο του αινίγματος του Mickaël Launay «Jungle Rhombique» από την Le Monde, 21/02/2026, με 25 κόμβους σε διάταξη ρόμβου:



Θα εμφανίσει στην έξοδο:

0: Alice at 10, Bob at 14

1: Alice at 15, Bob at 9

2: Alice at 16, Bob at 5

3: Alice at 12, Bob at 4

4: Alice at 8, Bob at 1

5: Alice at 5, Bob at 0

6: Alice at 4, Bob at 2

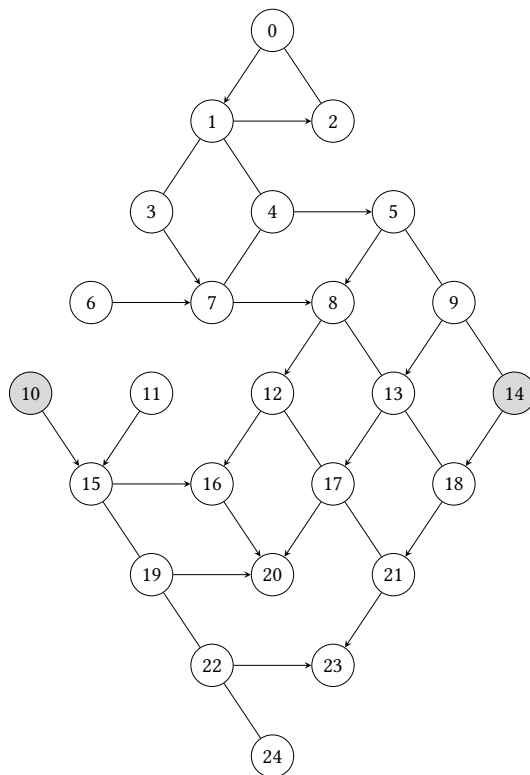
7: Alice at 1, Bob at 1

Meeting at node 1 at time step 7.

Αν ο χρήστης του προγράμματος δώσει:

```
python rendezvous.py -d graph_10.txt
```

τότε ο γράφος θα είναι ο παρακάτω κατευθυνόμενος:



Το πρόγραμμα θα εμφανίσει στην έξοδο:

No meeting is possible.

Adding 2 edges.

Adding 23 21.

Adding 23 17.

0: Alice at 10, Bob at 14

1: Alice at 15, Bob at 18

2: Alice at 19, Bob at 21

3: Alice at 22, Bob at 23

4: Alice at 23, Bob at 17

5: Alice at 21, Bob at 21

Meeting at node 21 at time step 5.

Καλή Επιτυχία!